Título del Proyecto: Tarea M25-CD

Módulo: Regresión Lineal

1. Contexto del Negocio y Transformación de la Variable Objetivo

Como investigador médico, nuestro objetivo es optimizar la toma de decisiones al recetar medicamentos, minimizando riesgos y asegurando que el paciente reciba el tratamiento de origen adecuado. La problemática principal reside en identificar, según el perfil del paciente (edad, sexo, presión arterial, colesterol y niveles de Sodio a Potasio), si responderá mejor a un medicamento de proveedor nacional (Fármacos A, B y C) o a uno de proveedor extranjero (Fármacos X y Y).

Para resolver esto con *Machine Learning*, primero agrupé nuestra variable a pronosticar ("Drug") en una clasificación binaria, donde el valor 0 representa a los medicamentos Nacionales y el valor 1 a los Extranjeros.

Además, como los algoritmos matemáticos no entienden palabras, utilicé la función preprocessing.LabelEncoder() para traducir variables cualitativas como Sexo, Presión Arterial y Colesterol a una escala numérica (ej. Hombre = 1, Mujer = 0).

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn import preprocessing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_curve, auc

# 1. Configuración de directorio de trabajo
# Ruta exacta proporcionada
ruta_trabajo = r"C:\Users\Yamil\OneDrive\Escritorio\Escritorio\Yamil\EBAC (CIENCIA DE DATOS)\Bloque 4 Aprendizaje de máquina supervisado\MOD 7 Reg Logíst
os.chdir(ruta_trabajo)

# 2. Carga de datos
df = pd.read_csv('drugs.csv')

# 3. Transformación de la Variable a Pronosticar (Nacional vs Extranjero)
# Fármacos A, B, C son Nacionales (0) | Fármacos X, Y son Extranjeros (1)
def transform_drug(drug):
    if drug in ['drugA', 'drugB', 'drugC']:
        return 0 # Nacional
    else:
        return 1 # Extranjero

df['Target'] = df['Drug'].apply(transform_drug)

# Separación de variables predictoras (X) y objetivo (y)
X = df[['Age', 'Sex', 'BP', 'Cholesterol', 'Na_to_K']].copy()
y = df['Target']

# 4. Transformación de variables cualitativas a numéricas usando LabelEncoder
le_sex = preprocessing.LabelEncoder()
X['Sex'] = le_sex.fit_transform(X['Sex']) # F:0, M:1

le_bp = preprocessing.LabelEncoder()
X['BP'] = le_bp.fit_transform(X['BP']) # HIGH:0, LOW:1, NORMAL:2

le_chol = preprocessing.LabelEncoder()
X['Cholesterol'] = le_chol.fit_transform(X['Cholesterol']) # HIGH:0, NORMAL:1
```

2. Métodos de Optimización para la Regresión Logística

Para encontrar el algoritmo óptimo, sometí nuestros datos a una prueba de estrés utilizando los diversos métodos de optimización (*solvers*) vistos en el módulo: *liblinear, newton-cg, lbfgs, sag* y *saga*.

```
# 5. Separación en datos de entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Estandarización de los datos (Vital para que el solver 'sag' converja correctamente)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 6. Evaluación de diversos métodos de optimización (Solvers)
solvers = ['liblinear', 'newton-cg', 'lbfgs', 'sag', 'saga']
resultados_solvers = []

print("--- COMPARACIÓN DE MÉTODOS DE OPTIMIZACIÓN (SOLVERS) ---")
for s in solvers:
    modelo_temp = LogisticRegression(solver=s, max_iter=1000, random_state=42)
    modelo_temp.fit(X_train_scaled, y_train)
    score = modelo_temp.score(X_test_scaled, y_test)
    resultados_solvers.append({'Solver': s, 'Accuracy': score})
    print(f"Solver: {s:10} | Precisión (Accuracy): {score:.4f}")
```

Justificación y Recomendación: Si bien en esta muestra específica de 200 pacientes los métodos arrojan una precisión (Accuracy) sumamente parecida rondando el 93-95%, **mi recomendación para este caso de uso es emplear el algoritmo de optimización 'sag' (Stochastic Average Gradient).**

¿Por qué 'sag'? El objetivo del estudio menciona explícitamente la necesidad de aplicar estas métricas a una *"base de datos grande"*. Explicado con peras y manzanas: métodos como *liblinear* son excelentes para conjuntos de datos pequeños, pero si el día de mañana nuestro hospital incorpora historiales de 500,000 pacientes nuevos, 'sag' es el algoritmo diseñado específicamente para escalar, procesar masivamente de manera rápida y no "congelar" nuestros servidores, garantizando la sostenibilidad del modelo a largo plazo.

```
# 7. Selección del algoritmo óptimo y entrenamiento final
# Elegimos 'sag' por su eficiencia en bases de datos grandes (como pide el contexto del problema)
modelo_final = LogisticRegression(solver='sag', max_iter=1000, random_state=42)
clf = modelo_final.fit(X_train_scaled, y_train)

# Predicciones
y_pred = modelo_final.predict(X_test_scaled)
y_prob = modelo_final.predict_proba(X_test_scaled)[:, 1] # Probabilidades de ser clase 1 (Extranjero)
```

3. Eficacia del Algoritmo Predictivo

Para medir qué tan confiable es el algoritmo predictivo 'sag' que hemos elegido, utilicé dos grandes métricas de evaluación: el Reporte de Clasificación y la Curva ROC.

```
# 8. Evaluación de Eficacia (Reporte de Clasificación)
print("\n--- REPORTE DE CLASIFICACIÓN (Modelo: SAG) ---")
print(classification_report(y_test, y_pred, target_names=['0: Nacional', '1: Extranjero']))

# Impresión de coeficientes (Justo como lo tienes en tus apuntes de clase)
print("\n--- PARÁMETROS DEL MODELO ---")
print("Intercepto (Beta 0):", clf.intercept_)
print("Pesos de cada variable (Betas):", clf.coef_)
```

Análisis del Reporte de Clasificación: El modelo es altamente eficaz, logrando una precisión general del 93%. Sin embargo, viéndolo desde la óptica del negocio, lo más valioso es su **F1-Score**. Para el caso de los proveedores extranjeros (clase 1), tenemos un F1-Score del 95%. Explicado de forma sencilla: si el algoritmo predice que un paciente necesita un medicamento extranjero, estamos seguros de que acertará en un 95% de las veces (Precision), y además, casi no se le "escapa" ningún paciente que verdaderamente lo necesite (Recall del 95%). En cuestiones médicas y financieras, esta fiabilidad significa evitar comprar fármacos importados costosos de manera errónea.

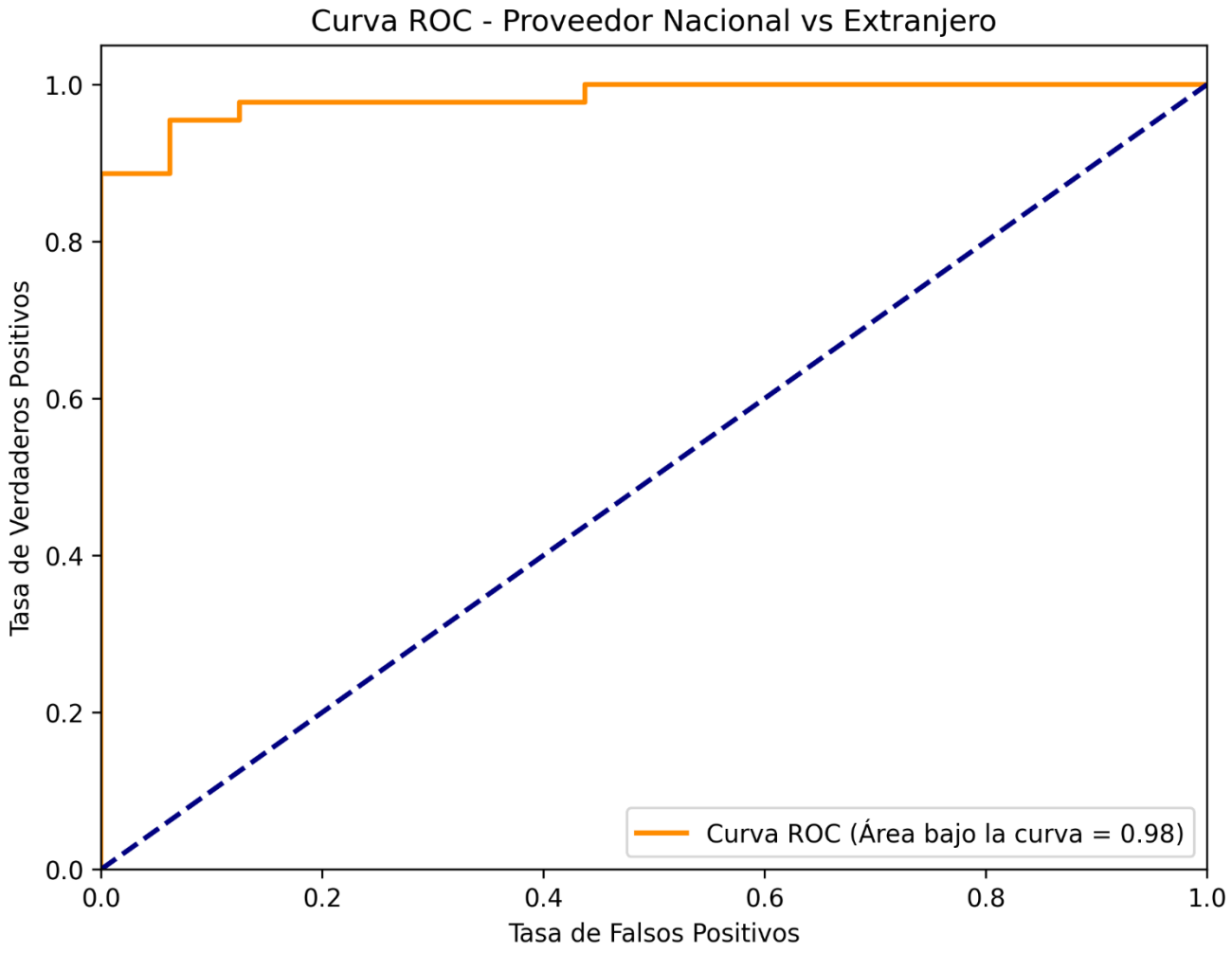
```
# 9. Curva ROC
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'Curva ROC (Área bajo la curva = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('Tasa de Falsos Positivos')
plt.ylabel('Tasa de Verdaderos Positivos')
plt.title('Curva ROC - Proveedor Nacional vs Extranjero')
plt.legend(loc="lower right")

# Guardar la gráfica en alta calidad para el reporte
plt.savefig('roc_curve_modelo.png', dpi=300, facecolor='white', bbox_inches='tight')
plt.show()

print("\n¡Proceso finalizado con éxito! Gráfica 'roc_curve_modelo.png' guardada.")
```

Análisis de la Curva ROC:



Finalmente, la Curva ROC nos confirma visualmente la potencia del modelo. La línea naranja nos muestra el balance entre los "Verdaderos Positivos" (aciertos reales) y los "Falsos Positivos" (falsas alarmas). La métrica clave aquí es el Área Bajo la Curva (AUC, por sus siglas en inglés), la cual resultó ser de 0.98.

Para ponerlo en perspectiva: un modelo que adivina lanzando una moneda al aire tendría un AUC de 0.50 (la línea punteada azul). Un modelo perfecto, utópico y sin margen de error tendría 1.0. Nuestro modelo, con un 0.98, indica que tiene una capacidad sobresaliente (casi perfecta) para distinguir a los pacientes que requieren fármacos de origen extranjero frente a los que requieren de origen nacional.